

Feature Flag Setup Guide

1. Environment

Backend: ASP.NET Core

Frontend: HTML5, CSS3, JavaScript

Containerization: Docker, Docker Compose

Deployment: AWS EC2

Logging: Serilog

2. Install Dependencies

Run this in the project root:

dotnet add package Microsoft.FeatureManagement.AspNetCore

This library enables feature flag evaluation and dependency injection support.

3. Configure Feature Flags

Add a new section to appsettings.json:

```
{
  "FeatureManagement": {
    "NewDashboard": false,
    "BetaSignup": true
  }
}
```

Then register Feature Management in Program.cs add:

using Microsoft.FeatureManagement;

builder.Services.AddFeatureManagement();

4. Use Feature Flags in Controllers example code

Inject IFeatureManager and use IsEnabledAsync():

```
using Microsoft.FeatureManagement;

public class DashboardController : Controller
{
    private readonly IFeatureManager _featureManager;

    public DashboardController(IFeatureManager featureManager)
    {
        _featureManager = featureManager;
    }

    public async Task<IActionResult> Index()
    {
        if (await _featureManager.IsEnabledAsync("NewDashboard"))
        {
            return View("NewDashboard");
        }
        return View("OldDashboard");
    }
}
```

5. Use in Razor Views example code:

```
@using Microsoft.FeatureManagement.Mvc
```

```
@addTagHelper *, Microsoft.FeatureManagement.AspNetCore
```

```
<feature name="BetaSignup">
```

```
    <button class="btn btn-success">Join Beta</button>
```

```
</feature>
```

```
<feature name="!BetaSignup">
```

```
    <p>Beta signups are currently closed.</p>
```

```
</feature>
```

6. Frontend Integration example code:

Expose feature flags with a simple API controller returning JSON.

Frontend JS can fetch /api/features and toggle UI elements accordingly.

```
[ApiController]
```

```
[Route("api/[controller]")]
```

```
public class FeaturesController : ControllerBase
```

```
{
```

```
    private readonly IFeatureManager _featureManager;
```

```
    public FeaturesController(IFeatureManager featureManager)
```

```
    {
```

```
        _featureManager = featureManager;
```

```
    }
```

```

[HttpGet]
public async Task<IActionResult> GetFlags()
{
    var flags = new Dictionary<string, bool>
    {
        ["NewDashboard"] = await _featureManager.IsEnabledAsync("NewDashboard"),
        ["BetaSignup"] = await _featureManager.IsEnabledAsync("BetaSignup")
    };
    return Ok(flags);
}
}

```

Then in JS:

```

fetch('/api/features')
.then(res => res.json())
.then(flags => {
    if (flags.NewDashboard) {
        document.getElementById("newDashboardSection").style.display = "block";
    }
});

```

7. Logging with Serilog example code:

Log when features are toggled or accessed:

```

Log.Information("Feature {FeatureName} is {Status}",
    "NewDashboard", await _featureManager.IsEnabledAsync("NewDashboard"));

```

8. Docker Integration example code:

Include your appsettings.json in the Docker image and use environment variables to override flags.:

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0
```

```
WORKDIR /app
```

```
COPY . .
```

```
ENV FeatureManagement__NewDashboard=false
```

```
ENTRYPOINT ["dotnet", "YourApp.dll"]
```

```
docker run -e FeatureManagement__NewDashboard=true yourapp:latest
```

You can toggle flags at runtime with:

```
docker run -e FeatureManagement__NewDashboard=true yourapp:latest
```